

Esperienza laboratoriale di fisica

Simulare un semaforo con Arduino

Indice

1	Introduzione e obiettivi	1
2	Richiami teorici	2
2.1	Arduino: hardware e software open-source	2
2.2	I componenti: LED, resistori e breadboard	2
2.3	Struttura di un programma Arduino	3
2.4	Comandi base	3
3	Scheda di laboratorio	5
3.1	Materiale occorrente	5
3.2	Domande preparatorie	5
3.3	Fase 1 — Un LED lampeggiante (Blink)	5
3.4	Fase 2 — Serie e parallelo: confronto della luminosità	6
3.5	Fase 3 — Il semaforo	7
3.6	Conclusioni	9

1 Introduzione e obiettivi

In questa esperienza useremo **Arduino** per costruire un **semaforo** con tre LED (rosso, giallo e verde). Partiremo dal circuito più semplice possibile — un singolo LED che lampeggia — e arriveremo passo dopo passo a programmare un semaforo ciclico.

Gli obiettivi sono:

- capire cos'è Arduino e come funziona la comunicazione tra software e hardware;
- conoscere i componenti base dell'elettronica: LED, resistori e breadboard;
- imparare la struttura di un programma Arduino (`setup()` e `loop()`);
- usare i comandi `pinMode()`, `digitalWrite()` e `delay()`;
- capire la differenza tra collegamento in **serie** e in **parallelo** osservando la luminosità di due LED;
- assemblare e programmare un semaforo funzionante.

2 Richiami teorici

2.1 Arduino: hardware e software open-source

Arduino è una piattaforma elettronica *open-source* composta da due parti:

- **Hardware:** una scheda con un microcontrollore (un piccolo computer) e una serie di pin digitali e analogici a cui collegare componenti esterni.
- **Software:** l'IDE (Integrated Development Environment) Arduino, un programma gratuito con cui si scrive il codice e lo si carica sulla scheda tramite cavo USB.

La scheda che useremo è l'**Arduino UNO**. I pin digitali numerati (0–13) possono essere configurati come **output** (la scheda invia corrente) o come **input** (la scheda legge un segnale). In modalità output digitale esistono solo due stati: **HIGH** (5 V, pin “acceso”) e **LOW** (0 V, pin “spento”).

2.2 I componenti: LED, resistori e breadboard

2.2.1 Il LED (diodo a emissione di luce)

Un **LED** è un componente che emette luce quando è attraversato da corrente nel verso giusto. Ha due terminali:

- **Anodo (+):** il terminale più lungo; la corrente entra qui.
- **Catodo (-):** il terminale più corto; la corrente esce qui.

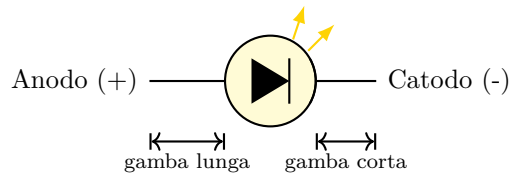


Figura 1: Schema del LED. La corrente deve entrare dall'anodo (+, gamba lunga) e uscire dal catodo (-, gamba corta). Se il LED è montato al contrario non si accende.

Attenzione. Non collegare mai un LED direttamente a 5 V senza un resistore in serie: la corrente sarebbe troppo alta e il LED si brucerebbe in pochi istanti.

2.2.2 Il resistore di protezione

Ogni LED ha una tensione di lavoro tipica ($V_{\text{LED}} \approx 2 \text{ V}$) e una corrente massima ($I_{\text{max}} \approx 20 \text{ mA}$). Il resistore in serie serve a **limitare la corrente** al valore corretto. Il suo valore si calcola con la legge di Ohm:

$$R = \frac{V_{\text{cc}} - V_{\text{LED}}}{I_{\text{LED}}} = \frac{5 \text{ V} - 2 \text{ V}}{0,015 \text{ A}} = 200 \, \Omega$$

Si usa di solito $R = 220 \, \Omega$ (il valore standard più vicino), che è sufficiente a proteggere il LED senza ridurre troppo la luminosità.

Il circuito base è quindi:

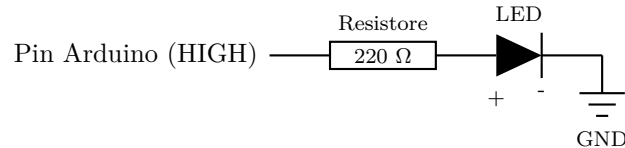


Figura 2: Circuito base: il pin digitale (in modalità HIGH = 5 V) alimenta il resistore in serie con il LED. Il catodo del LED è collegato a GND.

2.2.3 La breadboard

La **breadboard** è una basetta di plastica con una griglia di fori collegati internamente, che permette di costruire circuiti senza saldare. Le connessioni interne sono:

- le **righe centrali** (contrassegnate a-e e f-j): ogni riga di 5 fori è collegata internamente; tra le due metà c'è una separazione (il canale centrale).
- le **colonne laterali** (+/-): sono le rotaie di alimentazione; tutta la colonna + è collegata tra loro, idem per la colonna -.

2.3 Struttura di un programma Arduino

Ogni sketch Arduino ha due funzioni obbligatorie:

```
void setup() {  
  // Eseguita UNA SOLA VOLTA all'avvio.  
  // Qui si configurano i pin (input/output).  
}  
  
void loop() {  
  // Eseguita IN CONTINUAZIONE, all'infinito.  
  // Qui si scrive la logica del programma.  
}
```

2.4 Comandi base

Comando	Sintassi	Cosa fa
pinMode()	pinMode(pin, MODO)	Configura pin come OUTPUT o INPUT
digitalWrite()	digitalWrite(pin, VALORE)	Imposta pin a HIGH (5 V) o LOW (0 V)

Comando	Sintassi	Cosa fa
<code>delay()</code>	<code>delay(ms)</code>	Aspetta ms millisecondi (1000 ms = 1 s)

Nome e cognome: _____	Classe: _____	Data: _____
Componenti del gruppo: _____		

3 Scheda di laboratorio

3.1 Materiale occorrente

- scheda **Arduino UNO** + cavo USB;
- **breadboard** (mezza dimensione);
- **3 LED** (rosso, giallo, verde);
- **3 resistori da 220 Ω** ;
- cavi di collegamento (jumper maschio-maschio, vari colori);
- computer con **IDE Arduino** installato.

3.2 Domande preparatorie

Rispondete prima di iniziare.

D1. Qual è la differenza tra il pin lungo e il pin corto di un LED? Come va orientato nel circuito?

D2. Perché è necessario inserire un resistore in serie al LED? Cosa succederebbe se lo si collegasse direttamente a 5 V?

D3. Qual è la differenza tra `setup()` e `loop()` in un programma Arduino?

3.3 Fase 1 — Un LED lampeggiante (Blink)

3.3.1 Montaggio del circuito

Collegate un singolo LED (qualsiasi colore) al pin **digitale 12** di Arduino seguendo il circuito in figura: pin 12 $\rightarrow$ resistore 220 *Omega* $\rightarrow$ anodo LED $\rightarrow$ catodo LED $\rightarrow$ GND.

Verificate la polarità del LED prima di alimentare il circuito.

3.3.2 Caricamento del codice

Aprirete l'IDE Arduino, create un nuovo sketch e copiate il codice seguente:

```

void setup() {
  pinMode(12, OUTPUT); // pin 12 configurato come output
}

void loop() {
  digitalWrite(12, HIGH); // accende il LED
  delay(1000);             // aspetta 1 secondo
  digitalWrite(12, LOW);  // spegne il LED
  delay(1000);             // aspetta 1 secondo
}

```

Caricate lo sketch su Arduino (pulsante → con la freccia). Il LED dovrebbe lampeggiare con un periodo di 2 secondi (1 s acceso, 1 s spento).

3.3.3 Domande e modifiche

D4. Che cosa succede se cambiate entrambi i `delay(1000)` in `delay(200)`? Provate e descrivete il risultato.

D5. Scrivete qui sotto una versione del codice in cui il LED è acceso per 3 secondi e spento per 0,5 secondi:

D6. Spiegate con parole vostre cosa fa `pinMode(12, OUTPUT)`: perché è necessario e cosa succederebbe se si dimenticasse?

3.4 Fase 2 — Serie e parallelo: confronto della luminosità

3.4.1 Circuito in serie

Aggiungete un secondo LED allo stesso circuito inserendolo **in serie** al primo: pin 12 → R 220 *Omega* → LED₁ → LED₂ → GND. Usate lo stesso sketch della Fase 1.

Osservate la luminosità dei due LED e confrontatela con quella del singolo LED della Fase 1.

I due LED appaiono: ☐ più luminosi ☐ ugualmente luminosi ☐ meno luminosi

D7. Perché in serie i LED sono meno luminosi? (Suggerimento: pensate a come si divide la tensione tra i due LED.)

3.4.2 Circuito in parallelo

Rimuovete il secondo LED dalla serie. Aggiungete ora un secondo ramo **in parallelo** al primo: entrambi i LED hanno un proprio resistore da 220 *Omega* e sono collegati tra pin 12 e GND in modo indipendente.

I due LED appaiono: ☐ più luminosi ☐ ugualmente luminosi ☐ meno luminosi rispetto al LED singolo

D8. Perché in parallelo i LED hanno la stessa luminosità del LED singolo? (Suggerimento: che tensione si applica a ciascun ramo?)

3.5 Fase 3 — Il semaforo

3.5.1 Montaggio del circuito

Collegate i tre LED ai pin indicati, ciascuno con il proprio resistore da 220 *Omega*:

LED	Colore	Pin Arduino	Terminale +	Terminale -
1	Rosso	2	anodo → pin 2 tramite R	catodo → GND
2	Giallo	7	anodo → pin 7 tramite R	catodo → GND
3	Verde	12	anodo → pin 12 tramite R	catodo → GND

Verificate il circuito prima di procedere: ogni LED ha il proprio resistore in serie e i catodi sono tutti collegati alla rotaia GND della breadboard.

3.5.2 Caricamento del codice

Copiate il codice seguente nell'IDE Arduino e caricatelo:

```
// Pin dei LED
int pinRosso = 2;
int pinGiallo = 7;
int pinVerde = 12;

void setup() {
  pinMode(pinRosso, OUTPUT);
  pinMode(pinGiallo, OUTPUT);
  pinMode(pinVerde, OUTPUT);
}

void loop() {
  // ROSSO: stop
  digitalWrite(pinRosso, HIGH);
```

```

digitalWrite(pinGiallo, LOW);
digitalWrite(pinVerde, LOW);
delay(3000);

// ROSSO + GIALLO: preparazione al via
digitalWrite(pinGiallo, HIGH);
delay(1000);

// VERDE: via
digitalWrite(pinRosso, LOW);
digitalWrite(pinGiallo, LOW);
digitalWrite(pinVerde, HIGH);
delay(3000);

// GIALLO: preparazione allo stop
digitalWrite(pinVerde, LOW);
digitalWrite(pinGiallo, HIGH);
delay(1000);
// il loop ricomincia: torna al ROSSO
}

```

Osservate il semaforo in funzione e verificate che la sequenza sia corretta: Rosso → Rosso+Giallo → Verde → Giallo → Rosso → ...

3.5.3 Domande e modifiche

D9. Tracciate qui sotto la sequenza temporale del semaforo indicando quali LED sono accesi in ciascuna fase e per quanto tempo:

D10. Modificate i valori di `delay()` per ottenere tempi più realistici (es. rosso 5 s, verde 4 s, gialli 1 s). Scrivete qui le righe modificate:

D11. Nel codice, la variabile `pinRosso = 2` è definita all'inizio. Che vantaggio ha rispetto a scrivere direttamente il numero 2 in ogni `digitalWrite`?

3.6 Conclusioni

C1. Descrivete brevemente cosa fa ciascuno dei tre comandi usati in questa esperienza: `pinMode()`, `digitalWrite()`, `delay()`.

C2. Qual è stata la differenza di luminosità tra il collegamento in serie e quello in parallelo? Come lo spiegate usando la legge di Ohm?

C3. Elencate almeno **tre errori comuni** che potreste commettere durante il montaggio e spiegate come riconoscerli e correggerli.

C4. Pensate a un'estensione del progetto: come modifichereste il circuito e il codice per aggiungere un **pulsante** che, quando premuto, fa scattare il verde in anticipo (come il pulsante pedonale di un semaforo reale)?

Fine dell'esperienza